第四章 需求规格说明

2018年9月25日　9:20

- 需求规格说明的过程是迭代和增量的，将产生3种模型——状态模型、行为模型和状态变化模型

1. 体系结构优先权（architectural prerogatives）
    - **软件体系结构**定义了系统中相互作用的软件构件及**子系统**的结构及组织形式
    - 重点放在功能性需求上，非功能性需求可以暂时不做考虑
    - 所有软件建模的最重要目标是将构件**依赖**最小化

    a. 目标是开发一个能用好用的软件，利于软件开发

    b. 架构设计：层次化架构

        i. 模型model-视图view-控制器controller（MVC）框架

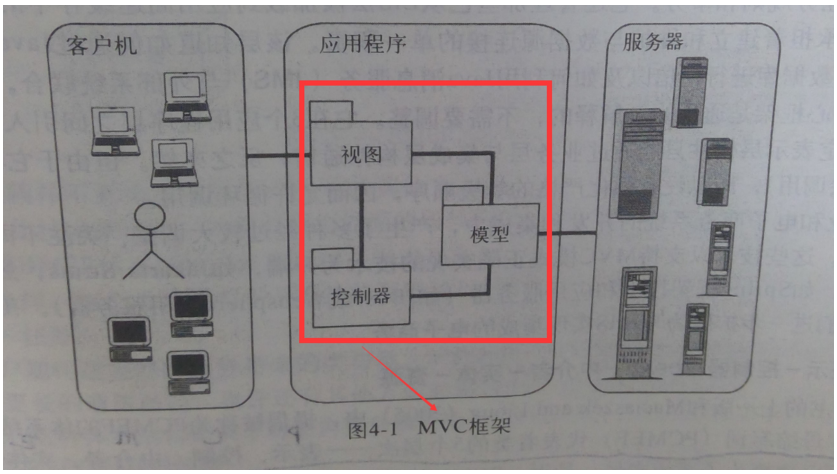
            □ MVC是**框架、元模型、编程环境**

            □ 给订阅者对象发送通知的过程称为**事件处理**

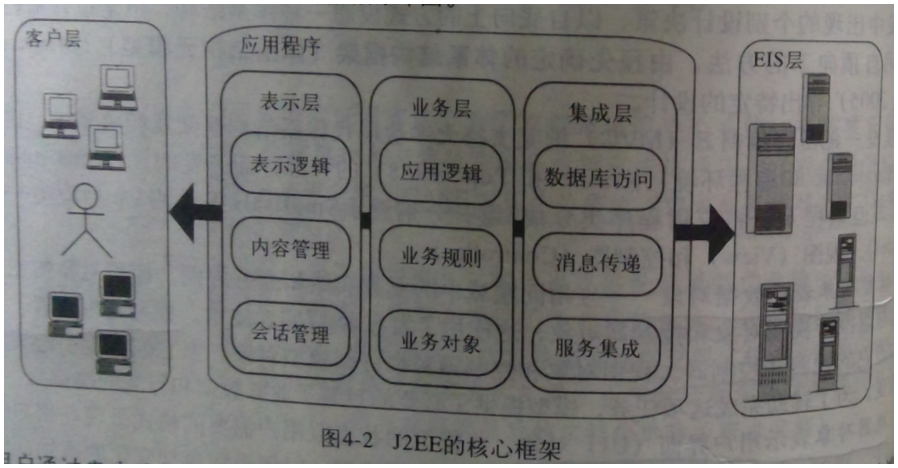
            1) 模型表示数据对象，模型是发布者，所以它不知道它的视图和控制器

            2) 视图对象表示用户界面（UI）对象

            3) 控制器对象表示鼠标



        ii. J2EE核心体系模型（以MVC为骨架的扩展）



- 最👈：用户通过客户层与系统交互，比如通过浏览器客户端
- 最👉：EIS层（资源层）是任意的持久信息传递系统，比如企业级数据库
- 用户通过表示层（web层或者服务器端表示层）来访问应用程序。表示层由用户界面代码和运行于Web服务器与/或应用服务器上的过程组成，参考MVC框架，表示层包括视图构件和控制器构件。
- 业务层包含表示层中的控制器构件还没有实现的一部分应用逻辑。它负责确认和执行业务范围内的业务规则和事务。
- 集成层承担着建立维护与数据源连接的单一职责，该层知道如何与数据库进行连接和通信。

iii. 表示-控制器-bean（实体数据）-中介者-实体（业务对象）-资源（PCBMER）

1）首字母缩写词代表着类的6个层次

2）该框架借用了J2EE核心框架中的外层（客户端层和EIS层）。在图中，各层表示为UML结点，带箭头的虚线表示依赖关系。如表示层依赖控制器层和bean层

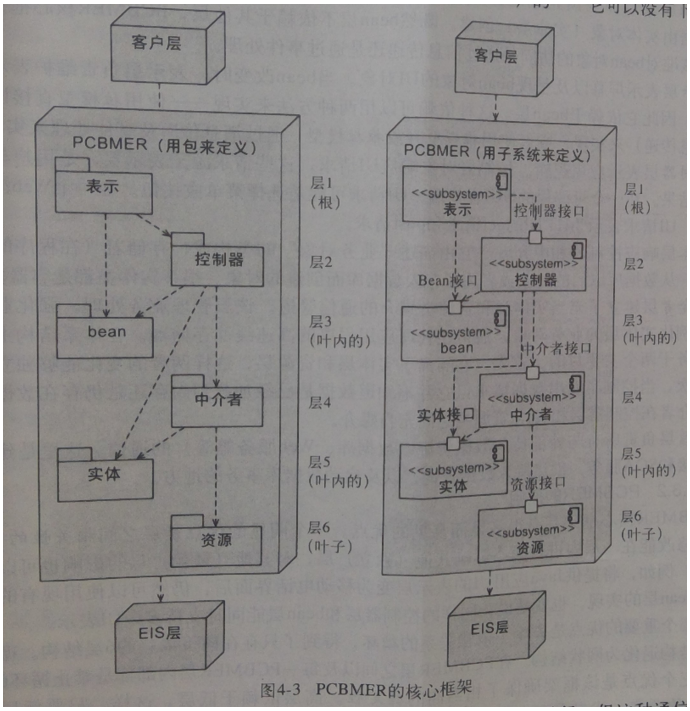


图4-3 PCBMER的核心框架

a) PCBMER核心框架的两种版本，一个用UML包定义，另一个用UML子系统定义。

b) 包是指定名字下的一组模型元素的集合，子系统结点常用来对大规模构件（子系统也是一种构件）建模，构件提供的服务被完全封装起来，暴露在外的是一组端口，这组端口定义构件的提供接口和依赖接口。

c) 6个层的具体表示

i) **bean层**表示那些预先确定要**呈现在用户界面上的数据类和对象**
the PCBMER layer representing the data classes and value objects that are destined for rendering on user interfaces

ii) ，除了用户输入外，bean数据由实体对象创建

iii) 表示层表示屏幕以及呈现bean对象的UI对象

iv) 控制层表示应用逻辑

v) 实体层响应控制器和中介者。它由描述“业务对象”的类组成

vi) 中介者层建立了充当实体类和资源类媒介的通信管道，它隔离了实体层和资源层，这样两者的变化能够独立的引入

vii) 资源层负责所有与外部持久数据（数据库、Web服务等等）的通信。

3）一系列原则

- PCBMER的优点
  - ◇ 层之间相关性的分离
  - ◇ 去除依赖关系的循环，得到了只存在向下依赖的6层结构
  - ◇ 确保相当高的稳定性

a) 向下依赖DDP

i) 自顶向下，高层对象依赖于低层对象，低层更稳定

b) 向上通知UNP，当下层发生变化

i) 促进层与层之间自底向上通信的低耦合

ii) 使用基于事件处理的异步通信来做到

c) 相邻通信NCP，相邻层之间可以通信

i) 每一层只能与有直接依赖关系的相邻层通信。

d) 显式关联原则NCP，如果两个类之间有关联必须显式标出
e) 循环去除CEP，避免环路依赖
f) 类命名原则CNP，类的名称前面加上一个字母表示包，如EVideo表示实体层的一个类
g) 相识包原则APP，6个层6个包

DDP – downward dependency principle
UNP – upward notification principle
NCP – neighbor communication principle
APP – acquaintance package principle
EAP – explicit association principle
CEP – cycle elimination principle
CNP – class naming principle

向下依赖原则	DDP	主依赖结构是自顶向下的。高层对象依赖于低层对象。低层比高层更稳定。
向上通知原则	UNP	促进了层与层之间自底向上通信的低耦合。高层对象担当订阅者，声明低层发生的变化。当低层对象改变其状态时，会给其订阅者发送通知。作为回应，订阅者同发布者通信。
相邻通信原则	NCP	每一层只能与有直接依赖关系的相邻层通信。
显式关联原则	EAP	建议在直接协作的类之间建立关联。
循环去除原则	CEP	循环依赖违反相关性分离方针，而且是复用的主要障碍。应将不受欢迎的类放进为此目的而特殊创建的一个新层/包中，或者强迫循环中的一条通信路径通过接口来通信。
类命名原则	CNP	通过类名就能了解该类属于哪个层/包。PCBMER 的每个类名前都加上层名的首个字母作为前缀。每个接口名前都加上两个大写字母作为前缀，第一个字母是 "I"，第二个字母表示所在的层。
相识包原则	APP	APP 是 NCP 的推论。相识包由对象在方法调用参数中传递的接口组成，而不是由具体的对象组成。

2. 状态规格说明——描述系统的静态结构
   - 识别系统中所有类及类与类之间的关系
   - 对象状态由一组属性值决定
   - 状态规格说明提供系统的结构（或静态）视图，主要任务是定义应用领域的类、属性以及与其他类的关系。类的操作一开始一般不考虑，将来从行为规格说明模型中导出
   - 首先识别实体类（业务对象），即定义应用领域的类和将在系统数据库中永久存在的类。
   - 至于服务于用户事件的类（控制器类），表示GUI呈现的类（表示类），以及管理GUI要表示的数据的类（bean类）等都将在后面定义

   a. 类建模（case工具），迭代增量式
      i. 发现类
         1) 名词短语方法——核心方法
            a) 候选类：
               i) 相关类revelant，类名的名词在需求文档中频繁出现
               ii) **模糊类**（只有名词短语法涉及）fuzzy，它们是最大的问题，要么划为相关类，要么划为无关类排除
               iii) 无关类irrelevant
            b) 要求需求文档完备且正确
         2) 公共类模式方法，将对象世界划分成有用的组
            a) 候选类分组
               i) 概念类
               ii) 事件类
               iii) 组织类
               iv) 人员类
               v) 地点类
            b) Just a guidance
            c) 只能松散地绑定到用户需求
         3) 用例驱动方法（IBM统一软件过程强调，有自底向上的特点）

a) 比名词短语法有些相似，效率高一些
b) 依赖于用例模型的正确性和完整性，加上叙述性描述、行为和交互模型作为补充

4) CRC类-职责-协作方法
a) 并不仅仅用来发现类，在头脑风暴阶段做类的获取
b) 职责是当前类为满足其他类所准备执行的服务（操作）
c) 协作者是需要履行的职责需要其他类的协作（服务）

5) Mixed混合方法
a) 根据领域知识大概猜一下初始类
b) 通用类模板法给一个框架
c) 名词短语法添加新类
d) 用例驱动的方法进行验证

6) 发现类的指南
◆ 分析员在分析类时，应该遵循下面的指南
a) 系统中每个类有清晰的目的陈述
b) 每个类是一组对象的模板陈述
◇ 单例类（一般情况下不考虑）
c) 每个实体类都应该有一组属性
i) OID或keys
d) 每个类应该有区别于其他类的属性
e) 类拥有一组操作

ii. 对类进行说明

1) 类的命名
a) 每个类给定一个名字，且不重名
b) 使用单数名词，每个实词首字母大写,如CourseOffering
c) 类名的长度有限制

2) 类的属性
a) 属性名用小写字母
b) 比如streetName或street_name

b. 关联建模（类之间的关系）最终类图不允许有孤立类
▪ 获取过程中尽量避免三元及以上、递归或环路关联，一般获取二元关联

i. 发现关联

ii. 说明关联の三要素

1) 关联的名字，一般情况下可以省略，当两个类之间超过一个关联同时存在时，必须有关联名
a) 关联的命名规则应遵循类名的约定——如Due
b) 关联角色的命名规则应遵循属性名的约定——小写字母开头后续单词首字母大写，角色名习惯以单词the开头——如theOrderLine

c)

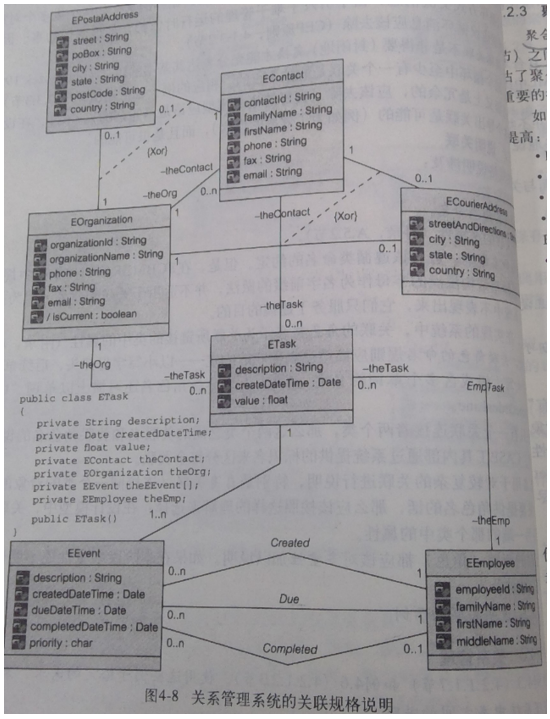


图4-8 关系管理系统的关联规格说明

2) 描述关联的角色，递归关联必须写，一般情况可不写

3) 确定多重性（必备）

c. 聚合和复合关系建模（整体部分的关系），复合强于聚合

i. 四种语义：

UML中的复合

1) ExclusiveOwns聚合，如书有章节
   a) 存在依赖性Existence-dependency，一个复合对象被删除，其构件也不复存在
   b) 传递性transitivity，C是B的一部分，B是A的一部分，则C是A的一部分
   c) 非对称性Asymmetricity，你是我的一部分，但我不是你的一部分
   d) 固定属性Fixed property，你是A1的一部分，你就不是Ai(i≠1)的一部分
2) Owns聚合,如车有轮胎，轮胎依赖于车存在，也可以独立存在，但单独一个轮胎意义不大
   a) 存在依赖性Existence-dependency
   b) 传递性transitivity
   c) 非对称性Asymmetricity

UML中的聚合

3) Has聚合，两者之间没有存在依赖，学院有系，没有某个系也可以活得很好
   a) 传递性transitivity
   b) 非对称性Asymmetricity
4) Member聚合，可以是多对多的多重性，e.g.Meeting has Chairperson

5)

语义存在依赖性传递性非对称性固有属性

	存在依赖性	传递性	非对称性	固有属性
Exclusive Owns	√	√	√	√
Owns	√	√	√	
Has		√	√	
Member				

ii. 发现聚合和复合

1) UML支持聚合和复合
   a) aggregation聚合是空心菱形，较弱的聚合
   b) composition复合是实心菱形，更强的聚合

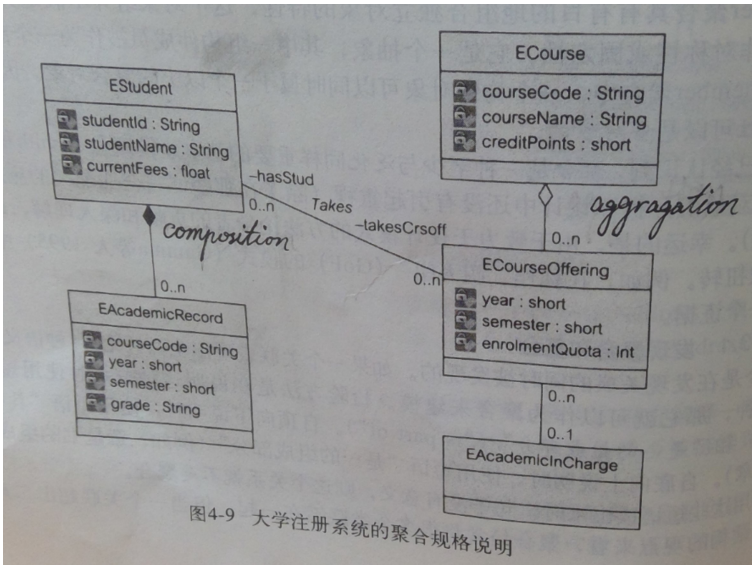


图4-9 大学注册系统的聚合规格说明

d. 泛化关系建模（父类子类的关系）
   - 泛化是将一般类（超类）与特殊类（子类）连接起来，泛化允许超类的特性被子类继承
   - 泛化的两个目的：
     - 可替换性：子类对象可替换父类变量
     - 多态性：同一个操作在不同子类里有不同的表现形式
   - 包含抽象方法的类——抽象类
   i. 发现泛化
      1) 比如is kind of/can be
   ii. 说明泛化
e. 接口建模，不是在需求分析阶段应该做的
   - 接口没有属性（除常量）、关联或状态。它们只有操作，并且所有操作都隐含是公共的和抽象的，在实现接口的类中，这些操作要被声明
   - 接口与类没有关联
   - 接口与另一个接口有泛化关系
   i. 发现接口
   ii. 说明接口
      1) 使用象征类的举行加上关键字<<interface>>表示，或者举行右上角的小圆圈，或者就是一个小圆圈，小圆圈下写接口的名字
      2) 一个类使用接口，用指向接口的一条虚线箭头表示，箭头上加构造型关键字<<use>>
      3) 一个类实现（提供）了接口，用一条末端带有三角形的虚线表示。该符号除了是虚线外，与泛化符号类似。可在虚线上加关键字<<implement>>

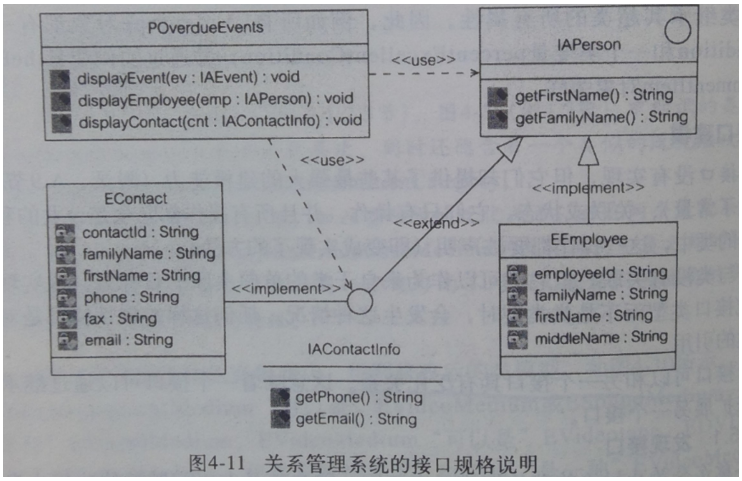


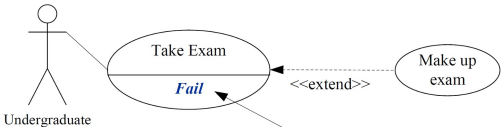
图4-11 关系管理系统的接口规格说明

f. 对象建模，用来描述复杂的类和类之间的关系，表示对象随时间变化的过程

3. 行为规格说明

- 描述系统操作视图，只定义关于系统冻结状态的操作视图，对象状态变化将在状态变更说明中描述
- 成果：完整用例图、活动图、时序图、完整（实体）类图
  - 对每一个用例都有时序图、通信图、活动图、以及描述此用例的类图
- 主要任务是定义应用领域中的用例，并确定这些用例执行中涉及哪些类，标识类操作和对象间的消息传递。

a. 用例建模，与类建模一样是迭代增量的
   i. 用例概念的本质特性
      1) 一个完整的功能
         a) 用例文档描述主流、子流、可选事件流
      2) 一个外部可见的功能（非内部功能）
      3) 一个正交功能（虽然在用例执行期间可以共享对象，但是每个用例的执行独立于其他用例）
      4) 与至少一个actor有关联
      5) 给参与者传递出确切值的一个功能（这个值在一个用例中获得）
   ii. 发现用例
      1) 需求文档中标识的需求
      2) 系统的参与者以及他们的使用目的
   iii. 说明用例
      □ 两个用例间总会存在**扩展**关系
      1) 四种常用关系
         a) 关联（参与者与用例间）
         b) 包含<<include>>，被包含用例一定执行
         c) 扩展<<extend>>，被扩展用例不一定被执行



         d) 泛化(通常是参与者的泛化)

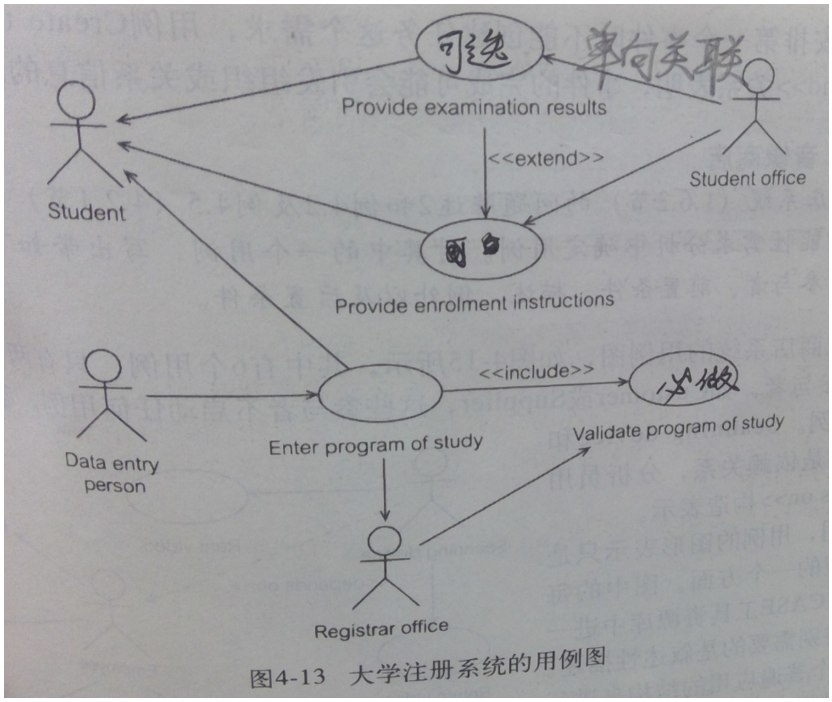


图4-13 大学注册系统的用例图

b. 活动建模（描述系统逻辑流程）
   - 活动图表示面向对象程序中的逻辑流（顺序控制、并发控制），可用来显示不同层次的计算细节
   - 活动模型对定义用例执行的动作流程特别有用
   i. 发现动作
      1) 每个用例可以有一个或多个活动图建模

2) 外部事件不在活动图中描述，如果外部中断频繁出现，应该使用状态机图
3) 在叙述性规格说明中带动词的短语都是候选动作

ii. 说明动作

1) 并发线程的发起（分叉）和汇合用同步线段表示
2) 可选线程用分支菱形来创建（分支）和合并

iii. 活动规格说明示例

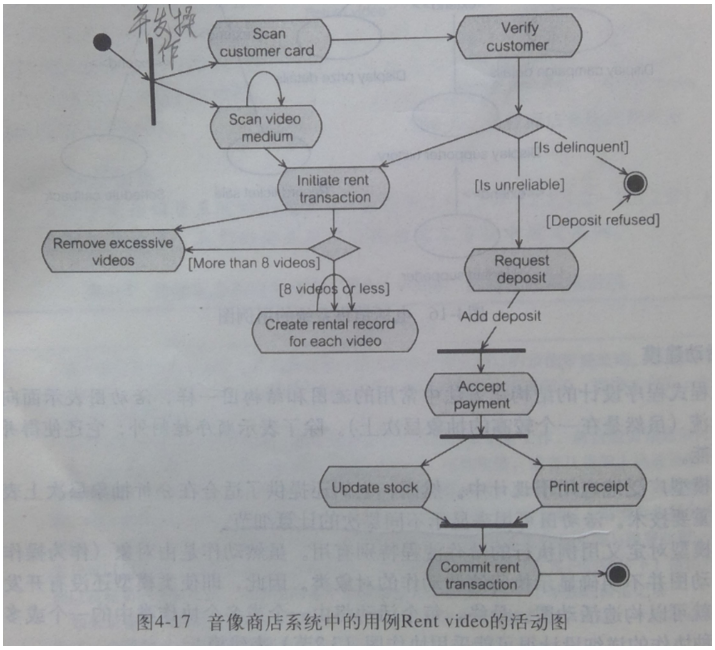


图4-17 音像商店系统中的用例Rent video的活动图

c. 交互建模

- 顺序图和通信图是两种交互图，
  - 顺序图在分析中更有用，按时间顺序展示对象之间的消息交换
  - 通信图在设计中更有用，描述消息和对象的交互关系

i. 消息序列

1) signal信号描述对象间的异步通信，消息的发送方在发送消息后可继续其他操作
   a) 如果消息没有返回参数，一般是异步asynchronous
2) call调用，描述对象间的同步synchronous操作，要将控制返回给发送者

ii. 序列规格示例

1) 集中式顺序图（控制类崩了，大部分就崩了）
2) 分布式顺序图

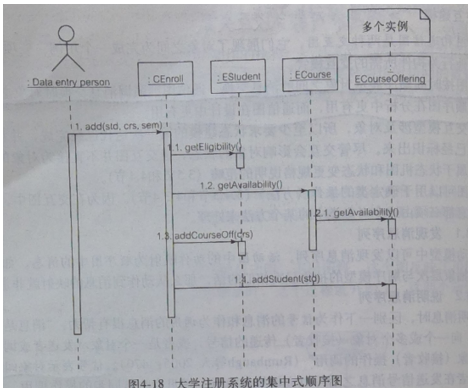


图4-18 大学注册系统的集中式顺序图

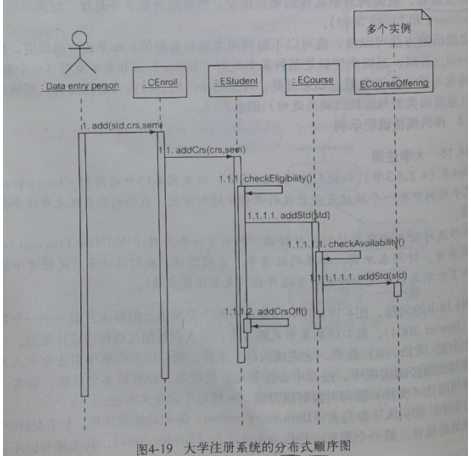


图4-19 大学注册系统的分布式顺序图

d. 操作建模
   - 类将一组操作作为服务提供给系统中的其他类，这组操作确定了类的公共接口，声明为公共可见性。

   i. 发现类操作
      1）从时序图中发现，每一条消息对应于类图中的方法
      2）从类或对象的基本功能的方法，如增删改查（CRUD），get,set方法

   ii. 操作规格说明示例

4. 状态变化规格说明，用在工程系统或者实时系统
   - 状态规格说明定义类的属性，行为规格说明定义类的操作
   - 在需求阶段的后期或者分析阶段将要结束时进行，主要是对异常的处理
   - 工具：状态机图（状态转换图）
   - 状态转换事件由以下事件触发（trigger）
     - 信号事件：建立对象间显式异步单向通信。
     - 调用事件：建立同步通信，调用者等待响应
     - 修改事件：**改变守卫值使守卫条件达到满足的事件**
     - 时间时间：特定状态下对象满足了一个时间表达式
   - 状态机规格说明示例，对象被事件激活，其中一些事件引起对象状态的改变

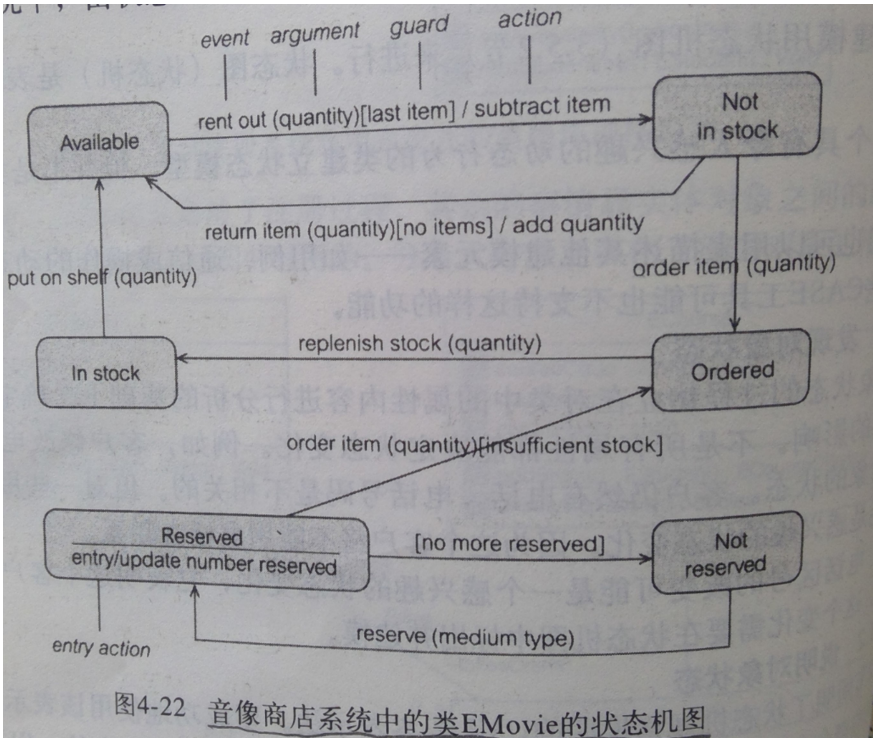


图4-22 音像商店系统中的类EMovie的状态机图